

Railway Scheduling: Incremental Solving and Pick Up and Delivery with Sequencing Constraints

Karl Osswald and Tim Leiber

University of Potsdam
karl.osswald@uni-potsdam.de
tim.leiber@uni-potsdam.de
2026-03-20

1 Introduction

Railway scheduling is a combinatorial problem that requires coordinating multiple trains on a shared infrastructure while avoiding conflicts and respecting operational constraints. In this project, we consider the railway scheduling problem as defined in the Flatland Challenge [6], using the Flatland environment¹. While this challenge is typically addressed using Reinforcement Learning (RL), we instead approach the problem using Answer Set Programming (ASP) – a declarative paradigm for solving combinatorial search problems – via the pre-existing toolkit². In Section 2 we introduce our basic encoding used for vehicle routing (VR) and focus on possible improvements regarding computational efficiency, particularly through incremental solving [5], allowing the planning horizon to be extended progressively and approximating an optimal solution rather than finding all possible answer sets thus reducing the cost of solving large instances. Section 3 introduces the Pick Up and Delivery Problem (PDP) with Sequencing Constraints, inspired by previous work on PDP and its variants with loading and sequencing constraints [1,2,4,7,8]. In particular, we present two alternative encodings that address PDP in the context of transporting passive vehicles, such as railway cars, which naturally follow last-in-first-out (LIFO) sequencing constraints. This version of PDP is relevant in railway operations, e.g. in the context of rolling stock management. Finally, in Section 4 we conclude on our findings and give an outline of possible future directions for research based on our results. Incremental solving can significantly enhance computational efficiency, while our comparison of two ASP encodings shows that the benefit of choosing a particular encoding depends on the presence of sequencing constraints: one performs better without such constraints, whereas the other is more efficient when sequencing constraints are enforced. Our source code can be found on the dedicated repository³.

¹ <https://flatland.aicrowd.com/intro.html>

² <https://github.com/krr-up/flatland>

³ https://github.com/ansovald/lokomotive_golm

2 Encoding Efficiency and Incremental Solving

2.1 Basic Encoding

In flatland, environments are 2D grids with different track types that allow predefined vehicle transitions. The toolkit also generates a number of agents with predefined start and end cell, as well as earliest departure and approximate arrival time, and an inverse speed. A global time frame is given to limit the maximum simulation time.

Cell Connections To approach the problem of vehicle routing, we first need to define the *connections* a specific track type allows:

`cell_conn(A, C0, In, C1, Out):`

If a train is in cell C0 facing in direction In, it can transition to cell C1 via action A and will then face in direction Out. Possible actions are `move_forward`, `move_left`, `move_right`, and `wait`, cf. fig. 1.

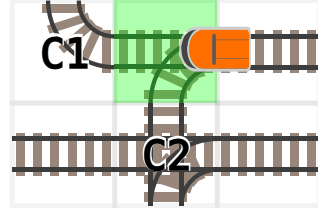


Fig. 1. The train moving into the marked cell C0 is facing west. If it moves *forward* into cell C1, it continues to face west; if it moves to the *left* into cell C2, it will face south.

Transitions Based on the cell connections, we define *transitions* that represent actual actions taken by the train at a certain time step:

`transition(ID, T0, C0, A, T1, C1, OutD):`

At time step T0, train ID is at cell C0.

Through action A, it reaches cell C1 at time step T1-1 and is then facing in direction OutD. At time step T1, it can choose the next transition.

To initialize a train, we define a *transition* that ‘moves’ the train onto its start cell at departure time. From there, consecutive transitions are chosen until the train reaches its destination using the following rule:

```
1 { transition(ID, T0, C0, A, T1, C1, OutD) :
    cell_conn(A, C0, Dir, C1, OutD),
    action_duration(A, S, Dur),
    T1 = T0 + Dur, global(MaxT), T1 <= MaxT } 1 :-
    transition(ID, _, _, _, T0, C0, Dir),
    speed(ID, S),
    not end(ID, C0, _).
```

This means that if a vehicle is in cell C0 at time step T0 facing in direction Dir, it can choose a transition according to the connections available at this cell. Action duration Dur is the inverse speed S for *movements*, and 1 for *wait* actions.

From the `transition` atoms, `action(train(ID), A, T)` are generated to ensure compatibility with the toolkit. As soon as a train reaches its target cell, an `arrived(ID, T)` atom is generated which is used to make sure all trains arrive at some point.

Collision Avoidance Intuitively, a collision occurs either if two trains face each other on the same train section and one or both of them move forward, or if they face in the same direction and the posterior train moves onto the cell occupied by the anterior. However, this assumption does not hold if trains with different speeds are involved: With a reverse speed > 1 , a vehicle can also be *between* cells at a given time step, cf. fig. 2. Also, in a real world application, it is a sensible safety measure to not allow a train to enter a section which another one is just in the process of vacating it.

We thus define the *occupies* predicate, which blocks both start and end cell during the whole duration of the transition, and add a constraint prohibiting that two trains occupy the same cell at the same time:

```
occupies(ID, T, (C0;C1)) :-
    transition(ID, T0, C0, _, T1, C1, _),
    T = T0..T1.
:- occupies(ID0, T, C),
   occupies(ID1, T, C),
   ID0 < ID1.
```

It must be noted that the original flatland toolkit assumes that a train only occupies one cell at any given time, regardless of speed. In some cases, our approach can make environments unsolvable in the given time frame.

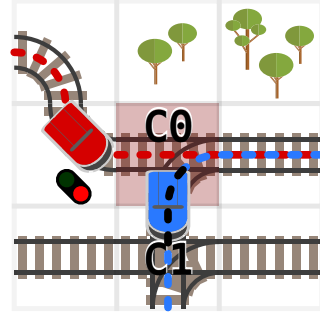


Fig. 2. The blue train is moving from C0 to C1 along the black dotted line with speed 2. The red train should only enter C0 once the blue train has arrived at C1, clearing the track (two time steps after it began moving).

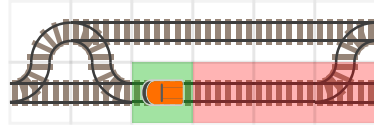


Fig. 3. If a train waits on the green cell, it effectively blocks all red cells up until the next junction

waits on the only assigned waiting cell on a longer track section, it effectively blocks the whole section until it starts moving again, cf. fig. 3.

Wait Restrictions A common method to reduce the search space when choosing train actions is to limit *wait* actions to certain cells, e. g., before junctions. As fig. 4 shows, unrestricted waiting does indeed require the solver to make on average 3.4 times more choices in the environments tested. In spite of that, we noticed that might make large environments with numerous agents unsolvable: If a train

Optimization In addition to simply finding solutions that guarantee all trains arrive, we also conducted experiments with minimizing the sum of arrival times of all trains. As fig. 4 shows, optimization is computationally much more challenging: in the most complex environment we were able to solve with optimization, the solution takes on the order of 10^6 choices, while the simple approach is on the order of 10^3 . It must be noted, however, that the advantage offered by wait restrictions diminishes considerably when optimizing for earliest arrival time.

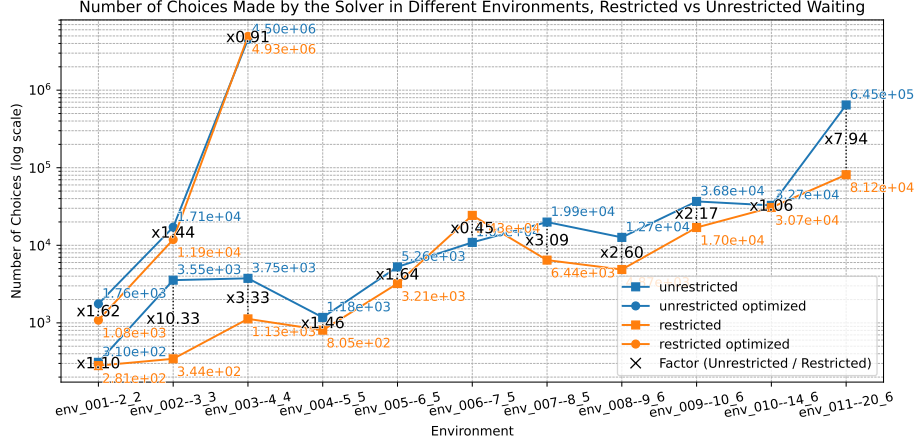


Fig. 4. Number of choices for an encoding with restricted vs. unrestricted waiting, without optimization and optimized for earliest arrival. Black numbers show the factor by which choices for unrestricted waiting are higher than for restricted waiting (on average: 3.2). Note that for optimized solutions, the advantage of restricted waiting diminishes considerably.

2.2 Incremental Solving

Since adding passive vehicles for pick up and delivery problems adds to the computational complexity, we experimented with incremental solving in order to boost the efficiency of our basic encoding.

Computational Complexity As we have seen in the previous section (p. 2), each train has up to four choices per time step. The upper bound for total number of choices for n trains and time frame T is thus $n \times \sum_{t=0}^T 4^t$, and the order of complexity accordingly $\mathcal{O}(4^T)$. Reducing the time frame by even one step thus removes up to $n \times 4^T$ choices during solving time.

Incremental Approach Following [5], we divide our encoding into four *sub-programs*. The **base** program contains the environment, all track definitions, cell connections, and train initializations. The **step(t)** program generates **transition** and **occupies** atoms for a given time step t (cf. sec. 2.1). The **check(t)** program contains the constraint that all trains must arrive at their target. Finally, the **optimize** program finds an optimal model according to the given optimization statements.

We implemented a custom **IncrementalFlatlandPlan** as a **clingo** application that first grounds the base program, and then, starting from time step 0, grounds all atoms up to the given time step and assigns the external atom **query(t)**. From time step 1 onwards, the **check** program is also grounded and

the prior `query(t)` released. This is repeated until a minimum time horizon is established at which all trains can reach their destination. Following that, we can optionally call the solver again with the `optimize` program.

To accommodate the incremental solving approach, we had to adjust our base encoding, specifically the `step` program: `occupies`, `action`, and `arrived` atoms must only be generated *for* the given time step `t`, even if a `transition` atom would allow for generating them for future time steps, e. g.:

```
occupies(ID, t, (C0;C1)) :-
    transition(ID, T0, C0, _, T1, C1, _), T0 <= t, t <= T1.
```

The main reason is that unlike in ASP normal programs, an incremental step must not reassign atoms that already exist.

2.3 Results

We tested our different encodings and approaches, with and without optimization, on 11 environments of increasing difficulty on the same machine with the same timeout, cf. fig. 5. The last two numbers in the environment names stand for the number of *agents* and the number of *stations*. Since optimization is computationally expensive, we did not get optimized results for all encodings and environments.

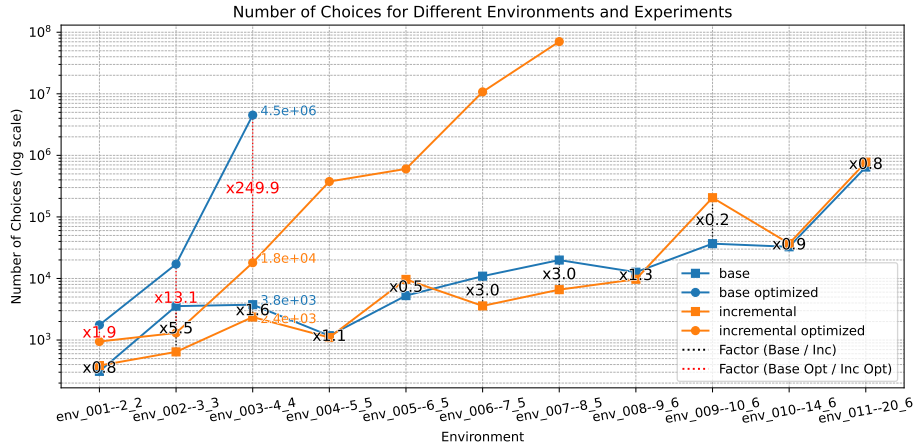


Fig. 5. Number of choices for base and incremental approach with and without optimization. On average, base approach needs 1.71 times more choices than the incremental approach, although the advantage appears to wane for more complex environments. Black dotted line shows the difference between optimization with base encoding vs. incremental approach for the most complex environment all encodings could solve in the allotted time.

As we can see in fig. 5, the base approach needs on average 1.71 times more choices than the incremental approach. However, it does not reduce complexity in all cases. This ultimately depends on the environment, and specifically the departure times of the trains: If a train departs comparatively late within the

given time frame, the incremental approach will still have to go through almost all time steps.

The performance gain becomes apparent, though, once we include optimization: the base encoding was only able to optimize the three most simple environments, while the incremental approach was able to optimize 7 environments. Judging from the trend seen for the first three environments, the gain appears to grow at least exponentially with environment complexity. For `env_003--4_4`, base approach needs 250 times more choices than the incremental approach, up from 13 for `env_002--3_3`.

Limitations and Outlook While we could establish a considerable advantage of the incremental approach for time-optimized solving, there are some limitations. Most notably, this approach only reduces the time horizon and might not be suitable for all optimization metrics. Furthermore, the `clingo` python module makes it possible to control the incremental part of the solving process, but not the optimization: as to our knowledge, it is not yet possible to retrieve solving statistics for each optimization step.

3 Pick Up and Delivery with Sequencing Constraints

3.1 Motivation and Formalization

Pickup and delivery problems arise in many transportation and logistics settings. In railway operations, a typical example is *rolling stock repositioning*, where railway cars are moved between locations to meet operational demand. More generally, such tasks fall into the class of vehicle routing and PDP, where vehicles transport items from pickup to delivery locations under operational constraints. Consequently, our approach may also apply beyond railway scheduling to more general PDP settings.

Pickup and delivery problems (PDP) have been widely studied in the context of vehicle routing and logistics with early work mostly considering the general PDP with multiple vehicles [3], while later research introduces variants with sequencing constraints such as FIFO or LIFO loading [1,4]. To this day there is wide interest in the topic of PDP with loading constraints [7,8]. Most existing approaches rely on algorithmic or statistical paradigms rather than declarative modelling. An advantage of our approach is that multiple variants of the problem can be implemented with relatively few lines of code. In contrast, a strictly algorithmic approach would typically require designing separate algorithms for each variant. Moreover, ASP supports a modular modeling style, allowing simpler problem variants to be obtained by disabling individual modules of the encoding rather than modifying the overall implementation.

Motivated by these applications and prior work, we extend the Flatland railway scheduling setting by introducing pickup and delivery tasks for passive vehicles such as railway cars. Trains act as carriers that collect cars at pickup locations and transport them to their destinations over the railway network. In

many railway settings, attached cars follow a last-in-first-out (LIFO) discipline, meaning that the most recently attached car must be detached first.

We consider two formal variants of this problem. The first requires that all cars must be delivered, while the second allows selecting a subset of cars to transport and optimizes the value of delivered cars.

All Vehicles Must Arrive Version Let $G = \langle V, E \rangle$ be a directed unweighted graph. For each train $t \in T$ and edge $e \in E$, let $x_{t,e} \in \{0, 1\}$ indicate whether train t traverses edge e . Each car $i \in C$ has a pickup vertex $s_i \in V$ and a destination vertex $d_i \in V$. The task is to compute routes for the trains such that every car i is transported from s_i to d_i , while respecting train capacity constraints and optionally LIFO sequencing constraints for cars attached to a train. A possible objective is to minimize the total distance traveled by all trains:

$$\min \sum_{t \in T} \sum_{e \in E} x_{t,e}.$$

Constraint-Based Optimization Version Let $G = \langle V, E \rangle$ be a directed unweighted graph and $x_{t,e} \in \{0, 1\}$ indicate whether train $t \in T$ uses edge $e \in E$. Each car $i \in C$ has a weight w_i and value v_i . Let $C_t \subseteq C$ denote the set of cars delivered by train t . The task is to determine train routes and transported cars such that train capacity constraints and optional LIFO sequencing constraints are satisfied. In contrast to the previous formulation, not all cars must be delivered. The objective is to maximize the total value of delivered cars:

$$\max \sum_{t \in T} \sum_{i \in C_t} v_i.$$

3.2 Encodings

To model the pickup and delivery extension, we implement two alternative ASP encodings that differ in how linking and unlinking operations between trains and cars are represented. The first encoding, *Batch-Linking Encoding* (BLE), allows multiple cars to be linked or unlinked within a single timestep. The second encoding, *Single-Operation Encoding* (SOE), restricts each train to at most one linking or unlinking operation per timestep. Both encodings build on the underlying train and track model and support optional LIFO sequencing constraints through a separate respective stack module. Hence they both are made of two modules, i.e. two ASP files named `moving_cars.lp` and `lifo.lp` containing the linking and stack logics respectively. If the programs are called without `lifo.lp` an instance is solved without sequencing constraints. Informations about the PDP related configurations of a setting such as the specific cars present in the map, their weights, values pick and and delivery spots are stored in the instance file via predicates.

The encodings also differ in how the order of cars is determined when LIFO constraints are applied. In BLE, multiple cars may be linked within the same

timestep, requiring an additional choice over the ordering of newly linked cars. In contrast, SOE restricts each train to a single linking or unlinking operation per timestep, so the stack order is implicitly determined by the sequence of linking timesteps and no additional ordering choices are required.

Both encodings support the *Constraint-Based Optimization Version* as well as the *All Cars Must Arrive Version* and are implemented as extensions on top of the existing encodings as described in Section 2. In the following, we focus on the *All Cars Must Arrive Version*, as the constraint-based optimization variant introduces additional computational complexity. Switching to the Constraint-Based Optimization Version however would be as easy as substituting a single line of code, namely a constraint rule for the optimization goal, i.e. maximizing the delivered value.

Batch-Linking Encoding (BLE) The Batch-Linking Encoding allows multiple cars to be linked or unlinked within the same timestep. Linking decisions are modeled by allowing trains to attach any number of cars that occupy the same cell as the train. There are two main choice rules in `moving_cars.lp` driving the linking and unlinking operations. Both rules are unbounded.

```
{ linked(TrainID, CarID, t) } :- t > 0,
    train_end_at(TrainID, t-1, C),
    car_state(CarID, C, t-1),
    unlinked(CarID, t-1),
    not car_target(CarID, C).
```

This rule allows a train to link any car that is currently located at the same cell, is not already attached to another train and has not yet reached its delivery spot. Since the rule is unbounded, zero or more cars may be linked simultaneously. The derived ternary predicate can be read as ‘at timestep t the car with identifier `CarID` is linked to the train with identifier `TrainID`.’

```
{ opt_unlink(CarID, t) } :- t > 0,
    linked(TrainID, CarID, t-1),
    train_end_at(TrainID, t-1, C), station(C).
```

Unlinking operations are modeled similarly. Whenever a train stops at a station, it may optionally detach any of its currently linked cars. Because these choices are unrestricted, BLE does not impose an upper bound on the number of linking or unlinking operations that can occur in a single timestep.

Additional rules take care of things such as the persistence of linking states, and rule out answer sets where a train’s capacity constraints, i.e. their maximum load is exceeded. Finally, an addition to the `check(t)` program inside `moving_cars.lp` crucially restricts answer sets to those satisfying the ‘all cars must arrive’ condition. Incremental grounding does not stop until at some timestep all cars are delivered:


```
:- query(t), num_cars(N),
   #count { CarID : car_arrived_by(CarID, t) } < N.
```

For more computationally complex instances one could also approximate a solution to the routing problem by demanding a fixed amount of delivered value. This would also be the rule which needs to be swapped when.

If LIFO constraints are enabled, cars attached to a train are treated as elements of a stack. When several cars are linked within the same timestep, their relative order cannot be inferred from time and must therefore be chosen explicitly. This is achieved by introducing stack positions for newly linked cars and assigning them non-deterministically via a choice rule:

```
1 { push_pos(TrainID, CarID, t, P) : pos(TrainID, t, P) } 1 :-
   new_link(TrainID, CarID, t).
```

Here, `push_pos(TrainID, CarID, t, P)` denotes that car `CarID` is inserted at stack position `P` on train `TrainID` at timestep `t`. This rule therefore determines the relative ordering of cars that are linked simultaneously.

The stack is then updated whenever linking or unlinking operations occur. In particular, the positions of previously attached cars are shifted according to the number of newly linked and removed cars:

```
stack_pos(TrainID, CarID, t, PosNew) :-
   stack_pos(TrainID, CarID, t-1, Pos),
   linked(TrainID, CarID, t),
   not new_link(TrainID, CarID, t),
   num_unlink(TrainID, t, K),
   num_new(TrainID, t, N),
   Pos > K,
   PosNew = Pos - K + N.
```

This rule ensures that cars are removed only from the top of the stack and that the remaining cars are shifted accordingly. Together with additional constraints preventing invalid pops, this enforces a LIFO discipline for unlinking operations.

Single-operation Encoding (SOE) The Single-Operation Encoding restricts each train to perform at most one linking or unlinking operation per timestep. In contrast to BLE, linking and unlinking are explicitly bounded, and operations are only allowed when a train is waiting at a station.

```
0 { linked(TrainID, CarID, t) :
   car_state(CarID, C, t-1),
   unlinked(CarID, t-1),
   not car_target(CarID, C) } 1 :-
   t > 0,
   transition(TrainID, _, _, wait, t, C, _),
   station(C).
```

This rule ensures that at most one car can be linked to a train at timestep t . Moreover, linking is only possible when the train is not moving, which introduces implicit waiting steps between operations. Unlinking is restricted in a similar way. In addition, only the car currently at the top of the stack may be removed when LIFO is enabled. As a result, SOE enforces a strict sequencing of operations, where linking and unlinking cannot occur simultaneously and must be separated by waiting steps. If LIFO constraints are enabled, the stack structure is significantly simplified compared to BLE. Since at most one car can be linked per timestep, the order of cars is implicitly determined by the temporal sequence of linking operations. Newly linked cars are always placed on top of the stack:

```
stack_pos(TrainID, CarID, t, 1) :-
    new_link(TrainID, CarID, t).
```

Existing cars are shifted accordingly when new cars are added or removed:

```
stack_pos(TrainID, OldCar, t, Pos+1) :-
    new_link(TrainID, _, t),
    stack_pos(TrainID, OldCar, t-1, Pos),
    not unlinked(OldCar, t-1).
```

```
stack_pos(TrainID, CarID, t, Pos-1) :-
    stack_pos(TrainID, CarID, t-1, Pos),
    Pos > 1,
    popped_from_train(TrainID, t).
```

In contrast to BLE, no additional choice over stack positions is required, as the ordering is uniquely determined by the sequence of operations.

3.3 Results

While BLE relies on a simplified treatment of sequencing, SOE allows for a more natural integration of pickup and delivery with sequencing constraints without requiring additional ordering choices. However, both encodings introduce a substantial number of choice points, as linking, unlinking, and stack operations must be explored across possible answer sets. As a result, instances with a low train-to-car ratio or extended time horizons quickly become computationally infeasible, since the approach requires exploring an extremely large space of possible schedules. A low train-to-car ratio can further increase complexity, as trains with limited capacity and unfavorable pickup and delivery locations may need to traverse the network multiple times, thereby extending the required time horizon and significantly enlarging the search space explored by the solver. Each leaf in the search space corresponds to a complete candidate solution (i.e., an answer set) representing a full system evolution.

We therefore simply provide a proof of concept and evaluate the two encodings with and without LIFO respectively on a set of controlled instances with a

specialized configuration, which allows us to illustrate their behavior and compare their performance under manageable conditions. In this context this means that pick up and delivery of cars are on the paths from start and end points of trains.

Without LIFO								With LIFO							
Vehicles	Batch Linking			Single Operation				Vehicles	Batch Linking			Single Operation			
	Easy	Med	Hard	Easy	Med	Hard			Easy	Med	Hard	Easy	Med	Hard	
3	0.63	0.67	1.288	0.729	0.774	0.816		3	1.463	1.333	1.553	0.916	0.915	1.025	
6	1.24	1.38	1.69	1.391	1.464	1.623		6	3.481	2.232	3.683	1.962	1.751	2.196	
9	1.64	1.81	1.759	1.713	1.916	2.032		9	3.946	4.778	5.296	2.666	2.298	3.456	

Table 1. Runtime comparison (seconds) of BLE and SOE encodings on 40×40 environments. Best and worst values are highlighted.

Table 1 shows runtimes for both BLE and SOE with and without LIFO. Easy instances mean that cars are spawned at train starting points, while in hard instances car and train starting points never coincide. In each instance, we consider a train-to-car ratio of 0.5, i.e. two cars per train.

Without LIFO constraints, SOE consistently outperforms BLE, whereas the opposite behavior can be observed when LIFO constraints are enabled, where BLE achieves better runtimes. Across all configurations, increasing the number of vehicles the imbalance between pickup and delivery locations leads to approximately linear scaling in these controlled environments, indicating that both only have a minor impact on performance in this setting. Truly impactful are train-to-car ratios and anything increasing the minimum time horizon.

As discussed previously, increasing the time horizon leads to exponential growth of the search space ($\mathcal{O}(4^T)$). In particular, reducing the number of available trains can significantly increase problem complexity, as multiple tours are required to deliver all cars. In such cases, it may be beneficial to relax the constraint in `check(t)` by introducing a lower bound on the delivered value rather than requiring all cars to be transported.

4 Conclusion and Future Directions

We extend the railway scheduling problem in ASP by combining incremental solving with an extension to pickup and delivery under sequencing constraints. Incremental solving can significantly improve computational efficiency, while our results demonstrate that encoding choice has a strong impact on performance: SOE performs better without sequencing constraints, whereas BLE is more efficient under LIFO constraints. Both approaches remain limited by the growth of the search space as the time horizon increases which is subject to properties of an instance such as the train-to-car ratio.

Regarding incremental solving, future work might entail extending the python module to call solving statistics on every step, and break the optimization once certain conditions are met and return the best model so far. Ending conditions could be established empirically: Experiments seem to indicate that each progressive optimization step tends to take longer than the prior one. Benchmarking on a range of environments of different complexity could provide a basis for ei-

ther estimating the optimum and then break the process once it reaches a certain threshold, or for establishing a breaking point where the number of choices and thus computational complexity exceeds the gains.

As another direction for future work, it is worth noting that the our encodings already capture a range of practically relevant problem variants. In particular, by interpreting car values as demand, the model naturally extends to scenarios such as passenger transport or demand-driven delivery. This suggests that the approach is not limited to railway freight settings, but can be applied to a broader class of transportation and logistics problems with and without sequencing constraints. A further direction for future work concerns dynamic settings such as vehicle re-routing in case of malfunctions, which are already supported by the underlying Flatland toolkit but were not accounted for in this work. Incremental solving is particularly well suited for such scenarios, as it allows efficient re-computation of solutions when parts of the environment change without solving the entire problem from scratch.

Appendix

References

1. Benavent, E., Landete, M., Mota, E., Tirado, G.: The multiple vehicle pickup and delivery problem with lifo constraints. *European Journal of Operational Research* **243**(3), 752–762 (2015). <https://doi.org/10.1016/j.ejor.2014.12.037>
2. Cheang, B., Gao, X., Lim, A., Qin, H., Zhu, W.: Multiple pickup and delivery traveling salesman problem with last-in-first-out loading and distance constraints. *European Journal of Operational Research* **223**(1), 60–75 (2012). <https://doi.org/https://doi.org/10.1016/j.ejor.2012.06.019>, <https://www.sciencedirect.com/science/article/pii/S0377221712004742>
3. Clarke, G., Wright, J.W.: Scheduling of vehicles from a central depot to a number of delivery points. *Operations Research* **12**(4), 568–581 (1964). <https://doi.org/10.1287/opre.12.4.568>
4. Erdogan, G., Cordeau, J.F., Laporte, G.: The pickup and delivery traveling salesman problem with first-in-first-out loading. *Computers & Operations Research* **36**, 1800–1808 (06 2009). <https://doi.org/10.1016/j.cor.2008.05.005>
5. Kaminski, R., Romero, J., Schaub, T., Wanko, P.: How to build your own asp-based system?! (2021), <https://arxiv.org/abs/2008.06692>
6. Mohanty, S., Nygren, E., Laurent, F., Schneider, M., Scheller, C., Bhattacharya, N., Watson, J., Egli, A., Eichenberger, C., Baumberger, C., Vienken, G., Sturm, I., Sartoretti, G., Spigler, G.: Flatland-rl: Multi-agent reinforcement learning on trains (2020), <https://arxiv.org/abs/2012.05893>
7. Pollaris, H., Braekers, K., Caris, A., Janssens, G.K., Limbourg, S.: Vehicle routing problems with loading constraints: state-of-the-art and future directions. *OR Spectrum* **37**(2), 297–330 (2015). <https://doi.org/10.1007/s00291-014-0386-3>
8. Sbai, I., Krichen, S.: Vehicle Routing Problems with Loading Constraints: An Overview of Variants and Solution Methods, chap. 1, pp. 1–23. John Wiley & Sons, Ltd (2022). <https://doi.org/10.1002/9781119902881.ch1>, <https://doi.org/10.1002/9781119902881.ch1>